MANUAL DEL PROGRAMADOR

Taller de Repaso e Integración en C++ — Actividad 6 (LPR)

E.E.S.T. N.º 1 "Eduardo Ader" — Vicente López

Estudiante: Malek Cheheid
Curso: 5° año, 3° "A"
Colegio: E.E.S.T. N.º 1 "Eduardo Ader", Vicente López
Materia: Laboratorio de Programación (LPR)
Profesor: Yamil Ganduglia
Actividad: N.º 6 — Taller de repaso e integración
Proyecto: repasogeneral
Versión: v1.0.0 (baseline aprobado)

Repositorio del proyecto

https://github.com/malekcheheid-art/repasogeneral.git

Documento orientado a quien mantiene, compila o amplía el código fuente.

1. Propósito del manual

Este manual describe la estructura del repositorio, el código fuente, las dependencias, el flujo de compilación y las tareas de mantenimiento del proyecto. Está pensado para el programador que necesita modificar o extender la suite de retos.

2. Requisitos de desarrollo

- Compilador g++ (TDM-GCC 10.3.0 o equivalente).
- Terminal PowerShell o VSCODE con extensión de C/C++.
- Git instalado (opcional pero recomendado para control de versiones).
- Sistema operativo Windows (línea de comandos PowerShell).

Si el equipo bloquea la compilación local, se puede usar OnlineGDB y conservar las capturas como evidencia.

3. Estructura del repositorio

```
repasogeneral/
+-- .gitignore
+-- LICENSE
+-- README.md
+-- docs/
|   +-- CHANGELOG.md
|   +-- manuales/
|   |   +-- manual_programador_v1.0.0.pdf
|   |   +-- manual_usuario_v1.0.0.pdf
|   +-- InformeEEST1_LPR2026_ACT06_G03_Informe_v1.0.0.pdf
|   +-- (pdf de la actividad)
+-- src/
|   +-- main.cpp
+-- capturas/
    +-- ejecucion_repasogeneral.png
    +-- traza_memoria.png
```

4. Descripción del código fuente

El programa se concentra en un único archivo: src/main.cpp. Allí se implementan los tres retos del taller.

Prototipos

```
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);
```

Reto 1 — sumaRecursiva

Función recursiva con caso base n <= 0. Devuelve la suma de 1 hasta n. Sin el caso base se produce Stack Overflow.

Reto 2 — Búsqueda secuencial

Recorre un arreglo de 10 enteros, compara celda por celda, imprime índice y dirección cuando halla el valor 18 y corta con break. Expone el concepto de memoria contigua.

Reto 3 — Intercambio con punteros

Recibe dos punteros int*, guarda el valor de *ptrA en una variable temporal, asigna *ptrB a *ptrA y luego el temporal a *ptrB. Modifica las variables originales por dirección.

Observación: no se validan punteros nulos. En un contexto escolar es aceptable, pero en una ampliación conviene agregar guards.

5. Compilación y pruebas

Comandos estándar en PowerShell (VS Code):

```
# Compilación
g++ src/main.cpp -o src/repasogeneral.exe

# Ejecución
.\src\repasogeneral.exe
```

Salida esperada: suma 15; valor 18 en índice [6] con dirección hexadecimal; x e y intercambiados (100/500 !’ 500/100).

Si el equipo bloquea la compilación local, usar OnlineGDB y guardar las capturas correspondientes como evidencia.

6. Control de versiones

El repositorio usa un esquema dual-remote: GitHub como origen principal y GitLab como respaldo. Comandos típicos:

```
git add .
git commit -m "mensaje descriptivo"
git push
```

Verificación de la configuración:

```
git remote -v
# origin  https://github.com/malekcheheid-art/repasogeneral.git (fetch)
# origin  https://github.com/malekcheheid-art/repasogeneral.git (push)
# origin  https://gitlab.com/malekcheheid-art/repasogeneral.git (push)
```

7. Mantenimiento y versionado

- Seguir SemVer/DocVer: MAJOR (alcance), MINOR (funcionalidad nueva), PATCH (correcciones).
- Registrar cambios en docs/CHANGELOG.md.
- Mantener el .gitignore para excluir .exe, .vscode/, *.out y archivos del sistema.
- Las evidencias (.png) se guardan en capturas/ (raíz del repositorio).
- No commitear ejecutables ni archivos temporales.

8. Tareas futuras sugeridas

- Generalizar sumaRecursiva con validación de entrada (n >= 0).
- Agregar manejo de punteros nulos en intercambiarValores.
- Convertir la búsqueda en función reutilizable.
- Ampliar la suite con vectores dinámicos o casos de prueba.